

C  puto Concurrente 2026

Pr  ctica 4

Spinlocks y algunas Primitivas

Profa: Gilde Valeria Rodr  guez

1 Contexto

Un Candado es un objeto que garantiza la exclusi  n mutua, sin embargo, en la pr  ctica   qu   hacemos si no pudimos obtener el candado? Hay dos opciones:

1. Esperamos activamente (Peterson, Bakery y Filter dan vueltas en un ciclo *while*)
2. Le pedimos al sistema operativo que agende otro hilo (Monitores: Candado Reentrante, Objeto *CountDownLatch*, Sem  foros, etc).

En esta pr  ctica nos vamos a enfocar en los candados que cumplen con el primer punto, a este tipo de candados les conocemos como Spinlocks, son objetos que permiten que los hilos esperen activamente hasta lograr entrar a la secci  n cr  tica.

Hasta ahora hemos visto implementaciones de Candados que tienen una relevancia te  rica, sin embargo, en la pr  ctica necesitamos eficiencia y lidiar con los hilos que no tienen   xito: lidiar con la **contenci  n**.

Contenci  n: ocurre cuando m  ltiples hilos intentan acceder al candado al mismo tiempo, una contenci  n alta implica que son muchos hilos al mismo tiempo y una contenci  n baja implica que solo son algunos.

Es imposible crear candados eficientes para cualquier n  mero de hilos sin operaciones primitivas (*testAndSet()*, *get()*, *compareAndSet()*), en esta pr  ctica se muestran ejemplos de candados que las utilizan. Son herramientas poderosas de sincronizaci  n, m  s poderosas que *volatile* (**Ojo:** No podemos utilizar candados porque estamos creando candados, no *synchronized*). En la teor  a veremos porque *compareAndSet()* es m  s poderosa que *testAndSet()* y que simples operaciones de escritura/lectura (*write()* o *read()*).

Todos los candados utilizan la instrucci  n *var.testAndSet(true)* (*getAndSet()*), esta instrucci  n reemplaza el valor de la variable *var* con *true* y devuelve el anterior. *BackoffLock()*, adem  s, utiliza una ventana de tiempo para aliviar la contenci  n. *MCSLock()* y *CLHLock()* incorporan una cola para, adem  s de determinar el hilo que puede acceder a la secci  n cr  tica, formar a los dem  s y as   distribuir a  n m  s la contenci  n.

2 Ejemplos

En el siguiente link: https://github.com/surindt/FC_CConcurrente/tree/main/Programas_P4

- Programa 1 (CounterAtomic): Contador linealizable no bloqueante, utiliza primitivas: getAndIncrement y get
- Programa 2 (RunSpin): Programa 2: Programa para medir el tiempo de los siguientes candados:
 1. Lock lock = new TASLock();
 2. Lock lock = new TTASLock();
 3. Lock lock = new BackoffLock();
 4. Lock lock = new MCSLock();
 5. Lock lock = new ALock(numberThreads);
 6. Lock lock = new ReentrantLock();
 7. Lock lock = new CLHLock();

3 Ejercicios

- Entrega en un pdf las respuestas de los ejercicios que no requieran implementarse y añade una breve descripción de los programas que entregas (sus nombres y qué hacen).
- Los programas deben ir en un zip. Debes realizar un programa en cada ejercicio que indique “Implementar”.
- Recuerda que debes utilizar Java 21 LTS.
- **Si no compila utilizando Java 21 LTS o si no se entrega la breve descripción de los programas se penalizará.**

Tiempo de elaboración: ≈ 2hr

Total de puntos: 100

Si ningún integrante en el equipo tiene una computadora con al menos 8 hilos comuníquese a la profesora.

La idea de las implementaciones de spinlocks (programas TASLock(), TTASLock(), BackoffLock(), MCSLock(), ALock(), CLHLock()) es aliviar la Contención distribuyendo a los hilos en distintas variables o dando tiempos fuera. Sin embargo, como investigaron la relación del *throughput* con el número de hilos en su práctica 2, el que ocupen un spinlock en un sistema dependerá del tamaño de las tareas, del hardware (sus compus), número de cores, la aplicación (si requieren Justicia o no).

1. Utiliza el programa *RunSpin* para probar cada uno de los spinlocks, cada integrante en el equipo debe completar la siguiente tabla: (**max-1** es el número máximo de hilos en tu compu menos uno)

Tabla comparativa cuando la Tarea es muy breve								
SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant	Counter Atomic
Tiempo de ejecución: 400 tareas con 4 hilos								
Tiempo de ejecución: 1000 tareas con 4 hilos								
Tiempo de ejecución: 1000 tareas con max-1 hilos								

2. Modificación del método `task(lock)`.

En lugar de incrementar un contador, modifica la función *task(lock)* para que los hilos trabajen con una matriz compartida de tamaño $n \times n$, donde $n = 10$. Cada hilo deberá asignar valores a la matriz dentro de una sección crítica protegida por *lock* y, una vez completada la asignación, imprimir su contenido. Implementa una función separada para la asignación y la impresión de la matriz. Cada integrante del equipo debe completar la siguiente tabla:

Tabla comparativa cuando la Tarea es más pesada							
SpinLocks / Counter	TAS	TTAS	Backoff	MCS	ALock	CLHLock	Reentrant
Tiempo de ejecución: 100 tareas con 4 hilos							
Tiempo de ejecución: 100 tareas con max-1 hilos							

- Escojan al integrante del equipo con mayor número de hilos. ¿Cuáles son las 2 implementaciones más rápidas del inciso 1? ¿A qué crees se debe?
- Escojan al integrante con mayor número de hilos. ¿Cuáles son las 2 implementaciones más rápidas del inciso 2? ¿A qué crees se debe?
- ¿Cuáles son las implementaciones que satisfacen la propiedad de Justicia? (Por el momento ignora *ReentrantLock*). Argumenta como lo hacen.